

# Every term in the project, explained once

Written for five AI students who did not arrive knowing combustion, and for the examiner who will ask one of them what knock is. Each entry says what the term means and why it matters *here*, which is the part a dictionary leaves out.

| TERMS     | MARKED ★               | GROUPED BY        | USE              |
|-----------|------------------------|-------------------|------------------|
| Around 90 | Must know for the viva | Where they appear | Sprint 0, week 2 |

0 · Using this

## Which of these you actually have to know

Roughly ninety terms is too many to memorise and the wrong goal anyway. Three tiers:

### ★ — ABOUT 20 TERMS

#### All five must know these

An examiner may ask any member. "That was someone else's part" is a bad answer in an individual defence. These are the week-2 teach-back material.

### UNMARKED — MOST OF THEM

#### Know where to look

Recognise the term, know roughly what it does, find the detail here when you need it. Nobody expects fluency on Woschni's correlation.

### LEAD-ONLY

#### Your own chapter

Whoever leads a phase should be fluent in everything in their section, including why alternatives were rejected.

A

## Combustion and knock

### ★ Knock also detonation, pinging

The unburned mixture ahead of the flame front — the **end gas** — gets hot and compressed enough to ignite *spontaneously, all at once*, instead of waiting for the flame to reach it. The result is a pressure wave crossing the chamber at the speed of sound, which hammers the piston crown and rings the block audibly. Sustained knock destroys pistons; severe knock destroys them in seconds. **This is the hardest constraint in the whole problem and the reason the project never touches a real vehicle's actuators.**

### ★ Spark advance °BTDC – degrees before top dead centre

How many crank degrees before the piston reaches the top of its stroke the plug fires. Flame propagation takes one to two milliseconds, so you must ignite early enough that peak pressure arrives just *after* top dead

centre, where the crank can convert it to torque. More advance means more torque up to a limit, and more knock risk always.

### ★ MBT minimum spark advance for best torque

The timing that maximises torque. Advance past it and pressure peaks while the piston is still rising, fighting the crank; retard from it and you waste the expansion. At high load you usually cannot reach MBT because the knock limit arrives first — **and that tension is what engine tuning fundamentally is.**

### ★ MFB50 crank angle of 50 % mass fraction burned, °aTDC

The crank angle by which half the fuel has burned. At MBT it lands at **8–10° after top dead centre** almost regardless of engine, speed or load — one of the most robust invariants in the field, and therefore the single best check that a combustion model is behaving. Our plant produces 10.5°, which is how we know the Wiebe parameters are sane.

### ★ Lambda $\lambda$

Actual air–fuel ratio divided by the stoichiometric ratio (14.7:1 by mass for gasoline).  $\lambda = 1$  is exactly enough air to burn all the fuel;  $\lambda < 1$  is **rich** (excess fuel);  $\lambda > 1$  is **lean** (excess air).

### ★ Enrichment

Deliberately running rich at high load. Thermodynamically wasteful — you inject fuel you cannot burn — but done anyway for three compounding reasons: liquid fuel absorbs its latent heat of vaporisation from the charge (**charge cooling**,  $\approx 25$  K in a direct-injection engine); the extra unburned fuel adds thermal mass; and with insufficient oxygen carbon leaves as CO, taking chemical energy with it. Net effect: **enrichment is a thermal-management tool**, and it is the fastest lever available when a turbine is approaching its limit.

### End gas

The unburned mixture ahead of the advancing flame. It is being compressed and heated by the flame's expansion, and it is the gas that autoignites when knock occurs.

### ★ Douaud–Eyzat induction-time integral

How we model knock. Autoignition is a *race*, not a threshold. At each instant the end gas has an induction time  $\tau$  — how long it would take to self-ignite if held at that pressure and temperature — given by  $\tau = 17.68 \cdot (0N/100)^{3.402} \cdot p^{-1.7} \cdot \exp(3800/T)$ . Conditions change constantly, so you accumulate the fraction used up:  $KI = \int dt/\tau$ . Knock occurs if KI reaches 1 before the flame consumes the end gas. Every dependency you would expect is in there — higher octane lengthens  $\tau$ , higher temperature crushes it exponentially.

### Wiebe function

An empirical S-curve describing what fraction of fuel has burned as a function of crank angle. It is what converts a spark timing into a heat-release schedule, which is what makes timing matter at all in the model.

### Octane number RON

A fuel's resistance to autoignition. Enters the knock correlation directly. Higher octane lets you run more advance before knocking.

### Residual gas fraction

The share of the cylinder contents left over from the previous cycle — hot exhaust that did not leave. It dilutes the fresh charge, raises its temperature, and slows combustion. Larger at low load.

### TDC / aTDC / BTDC

Top dead centre — the piston at the top of its travel; and angles measured after or before it. The reference point for all timing in the model.

### Slider-crank

The geometry converting piston motion to crank rotation. Gives the exact cylinder volume at any crank angle, which is what the pressure integration needs.

---

## B

## Work, torque and efficiency

### Mean effective pressure **MEP**

Work divided by swept volume, so it has units of pressure. The point of the normalisation is that it lets you compare engines of different sizes directly. Four flavours matter:

#### IMEP · indicated

Work done by the gas on the piston, from integrating pressure against volume around the closed cycle.

#### PMEP · pumping

Work spent breathing — pushing exhaust out and drawing intake in. Negative for a throttled naturally-aspirated engine (it sucks against a vacuum), *positive* under boost because the turbo does the pushing.

#### FMEP · friction

Rings, bearings, valvetrain, oil pump. Roughly 1–2 bar. Nearly constant while output is not, which is why part-load efficiency is poor. Modelled by the **Chen–Flynn correlation** as a function of peak pressure and mean piston speed.

#### ★ BMEP · brake

What survives at the crankshaft:  $\text{IMEP} + \text{PMEP} - \text{FMEP}$ . Torque follows directly. A modern 2.0 L turbo peaks near 20–22 bar BMEP.

#### ★ BSFC **brake specific fuel consumption, g/kWh**

Grams of fuel per kilowatt-hour — efficiency as a rate. About 235 g/kWh at a good engine's best point, far worse at idle. **Note the trap:** it divides by power, so it is undefined at zero torque — which is why our

reward uses fuel *mass flow* instead.

## Woschni correlation

The standard model for convective heat transfer from the burning gas to the cylinder walls. Load-dependent, which is why part-load efficiency is not overstated in our plant.

---

C

## Air path, boost and thermal

### MAP manifold absolute pressure, kPa

Pressure in the intake manifold. Below ~101 kPa is vacuum, above is boost. The primary load axis of every engine map.

### Turbocharger · wastegate · boost

Exhaust drives a turbine, which drives a compressor, which raises intake pressure. The **wastegate** bypasses exhaust around the turbine and therefore sets how much boost is produced. Its duty cycle is one of our agent's levers.

### Compressor map / flow ceiling

The limit of what a given compressor can actually deliver at a given speed. **Our plant currently lacks this** — manifold pressure is an input rather than something the turbo must produce, which is why peak power reads 16 % high. Adding it is the first task of the engine stage.

### IAT / ECT

Intake air temperature and engine coolant temperature. IAT matters enormously because it feeds the knock correlation exponentially.

### ★ EGT exhaust gas temperature

Gas temperature leaving the cylinder. Above roughly 950 °C, turbine blades and catalyst substrates are at risk. Rises with retard, falls with enrichment.

### Volumetric efficiency

How much air the engine actually ingests relative to its displacement at intake density. Peaks where the intake tuning is optimal, typically mid-range.

### ★ Lumped capacitance

Treating a physical mass as a single uniform temperature with one heat capacity:  $C \cdot dT/dt = \dot{Q}_{in} - UA \cdot (T - T_{amb})$ . Crude but exactly right for our purposes, because we care about *how fast heat accumulates*, not about temperature gradients within the metal.

### ★ Time constant $\tau = C / UA$

How long a thermal node takes to reach 63 % of the way to its new equilibrium. **The central quantity of this whole project.** Turbine housing  $\approx 60$  s; block and coolant  $\approx$  minutes; battery pack  $\approx 10$ – $40$  min. Preview is only worth something when your lookahead horizon is comparable to this number.

### Thermostat

A valve that bypasses the radiator below  $\sim 88$  °C. It is what makes coolant temperature a *regulated* variable rather than a free one, and why coolant sits at  $89$  °C until the cooling system runs out of capacity.

### Arrhenius

The exponential relationship between temperature and reaction rate:  $\text{rate} \propto \exp(-E_a/RT)$ . Used for our thermal damage model and for battery aging, because it captures the crucial fact that **10 °C hotter costs far more than 1 °C**. With  $E_a \approx 50$  kJ/mol it reproduces the classic "aging rate doubles per +10 °C".

---

D

## Vehicle data and diagnostics

### ECU / DME

Engine control unit. BMW calls theirs the DME (Digitale Motor Elektronik). The computer running the engine.

### OBD-II · CAN bus · PID

The diagnostic port under the dash, the internal network behind it, and the numbered parameter identifiers you request data by. **Read-only in this project** — nothing is ever written back.

### ★ Locked ECU

Refers to *write* access only — whether you can flash a modified calibration in. **Reading is unaffected.** The Supra's DME is OBD-locked, and it costs this project nothing.

### Knock retard

Timing the ECU has pulled in response to detected knock. Available as a manufacturer-specific channel. Note it is the controller's *response*, not knock itself — so it carries the ECU's own filtering and restore-rate behaviour with it.

### Fuel trims **STFT** / **LTFT**

Short- and long-term corrections the ECU applies to fuelling based on oxygen sensor feedback. Part of what makes a production ECU more than a bare lookup table.

### Direct injection **DI**

Fuel injected straight into the cylinder rather than the intake port. Means fuel evaporates inside the cylinder, which is what produces the  $\approx 25$  K charge cooling that enrichment exploits.

---

E

## Battery — for the second plant

### Equivalent circuit model **ECM**

Modelling a cell as a voltage source plus resistances and an RC pair, rather than simulating electrochemistry. The battery analogue of our zero-dimensional engine model: crude physics, right behaviour, fast enough to train against.

### OCV / SoC

Open-circuit voltage and state of charge. OCV is a known function of SoC for a given chemistry; the gap between OCV and terminal voltage under load is where the losses show up.

### C-rate

Current normalised by pack capacity. 1C fully discharges in one hour; 3C in twenty minutes. The natural unit for talking about charge and discharge stress.

### Calendar vs cycle aging

Batteries degrade with *time* (calendar, strongly temperature-driven) and with *throughput* (cycle, driven by current and depth of discharge). Both terms appear in our aging law, and the calendar term is what makes preview valuable in this plant.

### Preconditioning

Heating or cooling the pack *before* it is needed — ahead of a fast-charge stop or a hard climb. This is preview control in its purest form, and it is why the battery is the right second plant.

---

F

## Control theory — including the terms from the prior-art review

### ★ Preview control

Control that uses knowledge of a *future* reference or disturbance, not only present and past measurements. A mature field with decades of theory. Our project does not invent it — we measure how much it is worth.

### ★ $H_\infty$ "H-infinity"

A robust control design framework. The "H" is for **Hardy space**, a mathematical space of well-behaved transfer functions; the " $\infty$ " means you take the **supremum** — the worst case over all frequencies — rather than an average.

In plain terms: an  $H^\infty$  controller is designed to minimise the *worst-case* amplification from disturbance to error, across every frequency the disturbance might contain. That gives you a guarantee rather than an expectation, which is why it is the standard tool when robustness matters more than average performance. Contrast  $H_2$  control, which minimises the average energy response instead — better on typical inputs, no worst-case promise.

**Why it appears in our review:** the closest prior work to our claim is an  $H^\infty$  preview control result, so we have to know what the framework is in order to say precisely how we differ from it.

### ★ Regret

Borrowed from online learning. The performance gap between your controller and a benchmark that had *more information* than you did — here, an ideal non-causal controller that sees the entire disturbance in advance. "Regret goes to zero as preview grows" means a finite-preview controller approaches the omniscient one.

**This is essentially the question we ask**, which is why the Liu & Seiler paper is the nearest prior work — and why we must be precise that they prove convergence *exists* for linear systems without giving its *rate*.

### Spectral radius

The largest absolute eigenvalue of the closed-loop system matrix. For a stable discrete-time system it is less than 1, and it governs how fast the system forgets its past — making it **the discrete-time cousin of a time constant**. That is exactly why a regret bound decaying in the spectral radius is the linear-systems shadow of our  $H/\tau$  claim, and why we must distinguish ourselves carefully rather than dismiss it.

### LTI linear time-invariant

A system whose behaviour is linear and does not change over time. Nearly all classical control theory assumes it. **Our systems are not LTI** — knock limits and thermal limits are hard constraints and damage costs are exponential — which is precisely where the linear theory goes quiet and our contribution lives.

### Non-causal controller

One that uses future information. Physically impossible in general, but a useful theoretical benchmark: the best you could possibly do with perfect foresight.

### ★ MPC model predictive control

At each step, use a model to simulate forward over a horizon, optimise the action sequence, apply the first action, then repeat. The industry-standard way to use preview. In our project it is the **oracle baseline** — planning against the true plant with perfect preview to establish what is achievable.

### Prediction horizon

How far ahead an MPC controller plans. Standard practice sets it to roughly cover the open-loop settling time — which is the informal, folklore version of the criterion we are trying to measure properly.

## PI / PID controller

Proportional–integral(–derivative) feedback: act on the current error, its accumulation, and its rate of change. Used twice here — for the inner torque-tracking loop, and for updating the Lagrange multipliers.

## Setpoint

The target a controller is trying to hold. Our agent adjusts setpoints rather than driving actuators directly, which is what makes it *supervisory*.

---

G

## Reinforcement learning

### ★ MDP Markov decision process

The standard formalism: states, actions, transitions, rewards, and the Markov property — the future depends only on the present state, not the history.

### ★ CMDP constrained MDP

An MDP with cost functions and budgets alongside the reward. Maximise return *over the set of policies that satisfy the constraints*. **Why we use it:** some things should not have a price. "How much fuel is one missed newton-metre worth?" has no defensible answer, but scalarisation forces you to supply one as a weight. A constraint says the driver's demand is a requirement, and the optimisation happens in the space of ways to meet it.

### ★ Lagrange multiplier $\lambda$

Converts a constrained problem back into an unconstrained one, but with the trade-off rate *learned* rather than guessed. Read it as an adaptive penalty coefficient with a feedback loop: violate the constraint and  $\lambda$  climbs until the policy stops; satisfy it comfortably and  $\lambda$  decays toward zero. You never hand-tune a penalty weight again.

## PID Lagrangian

Updating  $\lambda$  with a PID controller on the constraint violation instead of plain gradient ascent. Plain ascent responds to *accumulated* violation, so it overshoots, the policy over-corrects,  $\lambda$  collapses, and training curves oscillate for millions of steps. Ten lines of code, problem gone.

### ★ SAC Soft Actor-Critic

Our algorithm. Off-policy continuous-control RL. An **actor** proposes actions, **twin critics** estimate their value, and an **entropy bonus** rewards the policy for staying stochastic. Right for us on three counts: continuous actions natively, sample-efficient because every transition is reused from a replay buffer, and the entropy term stops the policy collapsing prematurely onto one tuning strategy.

## Actor / critic / twin critics

The actor decides what to do; the critic estimates how good that was. Two independent critics are used and the *minimum* taken, which biases value estimates downward — safer than biasing up, and the standard fix for overestimation bias.

### ★ Entropy bonus $\alpha$

A reward for keeping the policy random rather than deterministic. Auto-tuned against a target entropy. In a landscape with a hard cliff like knock, this is exactly the behaviour you want — a deterministic policy that finds one good corner stops exploring everywhere else.

### Off-policy · replay buffer

Learning from transitions collected by an older version of the policy, stored and reused many times. Sample-efficient, which matters when every environment step costs a physics evaluation.

### Discount factor $\gamma$

How much future reward is worth relative to immediate. We use 0.98 at 5 Hz, giving roughly a 10-second effective horizon — deliberately matched to the turbine time constant rather than left at the habitual 0.99.

### ★ Reward shaping

Designing the reward so the behaviour you want is what actually scores highest. Where most of the real work goes, and where most silent failures originate.

### ★ Reward hacking

The agent finding a loophole — technically maximising your reward while doing something useless or absurd. Our original plan had one: terminating on *predicted* knock probability above 0.8 teaches the agent to sit at 0.79 forever, because that is where reward is highest and the episode never ends.

### Slew limit

A cap on how far an action may move per step. Without one, SAC produces policies that oscillate the full range every step — they score well and are physically absurd, and the plots make a thesis indefensible.

### Preference conditioning

Feeding the objective weights into the observation and resampling them each episode, so one agent learns a *family* of policies spanning the trade-off surface instead of a single point.

### Pareto front

The set of achievable outcomes where no objective can improve without another getting worse. What you get by sweeping the preference weights at evaluation.

### Scalarisation

Collapsing several objectives into one number with weights. What we deliberately do *not* do for torque tracking — see CMDP above.

### Gymnasium

The standard Python interface for RL environments (the maintained successor to OpenAI Gym). Forces your code into the shape every RL library expects.

---

H

## Learned models and uncertainty

### ★ Surrogate model

A fast learned approximation of a slow simulator. Ours is about a thousand times faster than the physics plant, which is what makes RL training tractable.

### ★ Model exploitation

**The most important failure mode in this project.** When an agent optimises against a *learned* model, it is not really searching for good actions — it is searching for actions the model *believes* are good. Wherever the model is optimistically wrong, that looks identical to a discovery, and the optimiser is specifically drawn there. Result: brilliant scores inside the surrogate, failure on the real plant. It is why we train on the surrogate and **evaluate on the physics**.

### Epistemic vs aleatoric uncertainty

Epistemic is uncertainty from *lack of data* — reducible by collecting more. Aleatoric is inherent randomness — irreducible. We measure epistemic uncertainty, because it is the kind that marks where the model is extrapolating.

### ★ Ensemble disagreement

Train several models on different initialisations and bootstrap samples, then measure how much they disagree. Inside the training distribution they agree closely; outside it they diverge. That divergence is our uncertainty signal, and it is how a model says *I don't know*.

### Pessimism $\beta$

The coefficient penalising ensemble disagreement in the reward. It tells the agent: *a gain you cannot verify does not count*. Push the policy back toward where the model can be trusted.

### Bootstrap sample

Sampling a training set with replacement so each ensemble member sees a slightly different dataset. One of the ingredients that makes members disagree usefully rather than identically.

### Held-out region

Deliberately excluding a whole area of the operating space from training, then querying it, to check the disagreement signal actually rises where the model is ignorant. **Verify this rather than assuming it** — an ensemble that is confidently wrong about the unseen makes the whole pessimism mechanism silently inert.

## Method and statistics

### ★ Ablation

Removing one component and re-running, to measure what that component contributed. **Our central experimental tool** — zeroing the preview channels is the ablation that answers the headline question.

### Counterfactual

What *would* have happened under a different action. The thing missing from stock vehicle logs, and the reason a physics model is needed to generate training data.

### ★ Causal identification

Whether the data can distinguish the effect of a variable from everything correlated with it. In a stock car, spark and lambda are deterministic functions of speed, load and temperature, so the action columns are collinear with the state columns and the effect of spark is **unidentifiable**. This is why more logging makes the problem worse rather than better — the single most important correction to the original plan.

### Collinearity

Two variables moving together so tightly that their separate effects cannot be told apart.

### Seed

The random number generator's starting value. Deep RL results vary enormously across seeds, which is why we run five per configuration and never report a single run.

### IQR **interquartile range**

The spread between the 25th and 75th percentiles. We report median and IQR rather than mean and standard deviation because RL return distributions are skewed and heavy-tailed, and a mean hides that.

### Latin hypercube sampling

A way of spreading sample points evenly across a multi-dimensional space, better than random sampling for the same budget. How we sweep the plant to generate surrogate training data.

### R<sup>2</sup>

Fraction of variance the model explains. Our surrogate acceptance gate is  $R^2 > 0.99$  on torque and fuel flow.

### Convexity · Jensen's inequality

A convex cost curves upward, so spreading a quantity over time costs less than concentrating it — the formal reason spreading a thermal load *should* help. We tested this as a hypothesis for why preview pays; **it did not hold as predicted**, and reporting that honestly is part of the work.

### Dimensionless number

A ratio with units cancelled, so it means the same thing in any system — the way a Reynolds number describes flow whether in a pipe or around a wing.  **$H/\tau$  is ours**, and being dimensionless is exactly what lets a result transfer from a combustion engine to a battery pack.

---

J

## Our own vocabulary

### ★ Plant

Control-engineering word for the thing being controlled. We have two: the engine and the battery.

### ★ Tier 0 / Tier 1 / Tier 2

Physics plant (defines truth) / surrogate ensemble (provides speed) / supervisory agent (our contribution).

**The rule that keeps the work honest:** train against Tier 1 because it is fast, score against Tier 0 because it is truth.

### ★ Preview horizon $H$

How far ahead the controller can see the upcoming disturbance. The independent variable of the whole thesis.

### ★★ $H/\tau$

Preview horizon divided by the time constant of the dominant constrained state. **The project's central claim is that this ratio predicts how much preview is worth**, independent of domain, actuators or units.

Measured evidence so far: holding physics and policies fixed and sweeping only  $\tau$  moved the preview advantage from 7.6 to 49.9 points.

### Dominant constrained state

The accumulating quantity whose limit the controller is actually defending. Turbine housing temperature on the engine; pack temperature on the battery. Choosing it correctly is what makes  $\tau$  meaningful — pick the wrong state and the criterion measures nothing.

### Preview advantage / preview edge

How much better the predictive policy does than the reactive one, in percentage points of damage reduction. Both use identical actuators and differ only in *when* they act.

### Baseline-relative reward

Scoring the agent on its fractional improvement over the production controller evaluated at the same operating point. Makes every reward term dimensionless and near unit scale, so the preference weights genuinely control the trade-off — and gives a reward of zero a precise meaning: *identical to the production calibration*.

### Damage rate

An Arrhenius-style accumulated cost on turbine and oil temperature, rather than a threshold. Gives gradient everywhere, and correctly makes 10 °C hotter much worse than 1 °C.

### **Constraint budget $d_i$**

How much of a constraint the policy is permitted to violate in expectation. Torque tracking gets 1 % of the cycle; knock gets 0.1 %.

### **Neutral action**

The trim vector that reproduces the baseline controller exactly. Its reward should be  $\approx 0$  — the acceptance test for the environment, and measured at  $-0.005$ .

---

**The twenty terms marked ★** are the week-2 teach-back list. Every member should be able to explain each of them to someone outside the project in about a minute, regardless of which phase they lead.

**The three that carry the most weight:** *knock*, because it is the constraint everything else defends; *model exploitation*, because it is the failure mode that would invalidate every result; and *H/ $\tau$* , because it is the contribution.